

Самостійна робота №3: Тестування React-компонентів

Тема: Тестування React-компонентів

Мета: Вивчити підходи до тестування React-застосунків, навчитися писати unit-тести для компонентів та хуків за допомогою Jest та React Testing Library.

Кількість годин: 4

Теоретичний матеріал

Підходи до тестування

Тестування фронтенд-застосунків поділяється на рівні: **unit-тести** (окремі функції, компоненти), **інтеграційні тести** (взаємодія між компонентами) та **E2E-тести** (повний сценарій користувача). React Testing Library фокусується на тестуванні поведінки компонентів з точки зору користувача, а не деталей реалізації.

Jest

Jest - фреймворк для тестування JavaScript від Meta. Основні функції: `describe()` - групування тестів, `test()` або `it()` - окремий тест, `expect()` - перевірка результату (assertions). Матчери: `toBe()`, `toEqual()`, `toContain()`, `toThrow()`, `toHaveBeenCalled()`. Jest підтримує моки (`jest.fn()`, `jest.mock()`), таймери (`jest.useFakeTimers()`), снапшот-тестування (`toMatchSnapshot()`).

React Testing Library (RTL)

RTL пропонує утиліти для рендерингу компонентів та взаємодії з ними у тестах. Основна функція - `render()`, яка рендерить компонент у віртуальний DOM. Запити для пошуку елементів: `getByText()`, `getByRole()`, `getByLabelText()`, `getByPlaceholderText()`, `getByTestId()`. Запити бувають трьох типів: `getBy` (синхронний, кидає помилку), `queryBy` (повертає null), `findBy` (асинхронний, повертає Promise).

Взаємодія з компонентами

Бібліотека `@testing-library/user-event` моделює реальні дії користувача: `userEvent.click()`, `userEvent.type()`, `userEvent.selectOptions()`. Для перевірки стану DOM використовуються матчери з `@testing-library/jest-dom`: `toBeInTheDocument()`, `toHaveTextContent()`, `toBeVisible()`, `toBeDisabled()`.

Тестування хуків

Хуки тестуються за допомогою `renderHook()` з `@testing-library/react`. Функція повертає `result.current` - поточне значення хука. Для оновлення стану використовується `act()`. Приклад: тестування кастомного хука `useCounter` з перевіркою початкового значення, інкременту та декременту.

Мокінг

Для ізоляції компонентів від зовнішніх залежностей використовуються моки. `jest.mock('module')` замінює модуль на мок-версію. API-запити мокуються через `jest.mock('axios')` або бібліотеку MSW (Mock Service Worker). Мокінг дочірніх компонентів дозволяє тестувати батьківський компонент незалежно.

Контрольні питання

1. У чому філософія React Testing Library? Чому вона рекомендує тестувати поведінку, а не реалізацію?
2. Яка різниця між запитам `getBy`, `queryBy` та `findBy` у RTL?
3. Чому `getByRole` вважається пріоритетним запитом і як він пов'язаний з доступністю?
4. Як протестувати асинхронний компонент, який завантажує дані з API?
5. Що таке `act()` і коли його потрібно використовувати?
6. Як протестувати кастомний хук за допомогою `renderHook()`?
7. Які переваги та недоліки снапшот-тестування?

Практичне завдання

Напишіть тести для компонента `LoginForm`, який містить поля `email` та `password`, кнопку "Увійти" та виводить помилку при невалідних даних:

1. Тест: форма рендериться з усіма необхідними полями.
2. Тест: кнопка "Увійти" заблокована, коли поля порожні.
3. Тест: виводиться повідомлення про помилку при невалідному email.
4. Тест: викликається `onSubmit` з правильними даними при успішній валідації.

Порядок виконання

1. Вивчити теоретичний матеріал, наведений у цьому документі.
2. Ознайомитись з рекомендованими джерелами.
3. Відповісти на контрольні питання.
4. Виконати практичне завдання.

Рекомендовані джерела

- [React Testing Library - документація](#) - офіційна документація RTL
- [Jest - документація](#) - офіційна документація Jest
- [Testing Library - запити](#) - пріоритет та використання запитів
- [Common mistakes with RTL](#) - типові помилки при тестуванні (Kent C. Dodds)
- [jest-dom matchers](#) - додаткові матчери для DOM
- [MSW - Mock Service Worker](#) - мокінг API-запитів